

Schema Validity Is Not Enough: Zero-Trust Evaluation of Local SLMs for Industrial Robot Task Planning with Microsoft Foundry Local

Adhish Rao

University College London

MSc Systems Engineering for the Internet of Things

United Kingdom

Abstract

This dissertation investigates whether local small language models (SLMs), served through Microsoft Foundry Local, can support industrial robot task-planning workflows when their outputs are treated as untrusted action proposals rather than executable commands. The project is positioned in industrial robotics and manufacturing automation and evaluates a zero-trust validation framework that separates schema validity from semantic validity, safety validity and execution eligibility. The current evidence base is drawn from a sequence of prototypes, with Prototype 5 acting as the final evidence orchestration and reporting layer. The work demonstrates, within a bounded benchmark and defined validation policy, that schema-valid model outputs can still be semantically incorrect, unsafe or ambiguous, and therefore require deterministic validation before downstream execution.

Keywords

Microsoft Foundry Local, small language models, industrial robotics, zero-trust validation, task planning, local AI, execution eligibility

7.6	Zero-Trust Gating and False-Accept Prevention	4
7.7	Local-vs-Cloud Deployment Trade-Off	4
7.8	Resource Profiling	4
7.9	Safety-Latency Frontier	4
7.10	Summary Against Research Questions	4
8	Discussion	5
9	Alignment with the Microsoft IXN Brief	5
10	Industrial Robotics Deployment Context	5
11	Limitations and Claim Boundaries	5
12	Future Work	5
13	Conclusion	6
14	Conclusion	6
	References	7
A	Verification Evidence	7
	References	7

CONTENTS

Abstract	1
Contents	1
List of Figures	1
List of Tables	1
1 Introduction	2
1.1 Project Overview and Current Evidence Baseline	2
1.2 Research Aim, Objectives and Questions	2
2 Background and Related Work	2
3 System Design and Methodology	2
3.1 System Architecture	2
3.2 Layer 1: Local Inference	2
3.3 Layer 2: Zero-Trust Governance	3
3.4 Layer 3: Optional Execution-Context Visualisation	3
3.5 Evidence Logging	3
4 Methodology and Evaluation Framework	3
5 Prototype Evolution and Evidence Contribution	4
6 Results and Evaluation	4
7 Results Overview	4
7.1 Foundry Local Inference Feasibility	4
7.2 Structured-Output Reliability	4
7.3 Schema Validity Versus Execution Eligibility	4
7.4 Ambiguity and False Accepts	4
7.5 Model Family and Size Trade-Offs	4

LIST OF FIGURES

1	Local-first zero-trust evaluation architecture for industrial robot task-planning proposals.	3
---	--	---

LIST OF TABLES

1	Research questions mapped to detected evidence.	3
2	Compressed metric taxonomy used in the evaluation framework.	5
3	Prototype contribution and evidence status.	5
4	Summary alignment with the Microsoft IXN brief.	6
5	Claim boundaries for dissertation interpretation.	6

1 Introduction

1.1 Project Overview and Current Evidence Baseline

This dissertation, titled *Schema Validity Is Not Enough: Zero-Trust Evaluation of Local SLMs for Industrial Robot Task Planning with Microsoft Foundry Local*, investigates whether local small language models (SLMs) can support industrial robot task-planning workflows when their outputs are treated as untrusted action proposals rather than executable commands. The central claim is deliberately bounded: within the evaluated benchmark and under the defined validation policy, local SLMs can contribute to robot task planning only when deterministic validation mediates every model-generated proposal before any downstream execution context is reached.

The work is positioned in industrial robotics and manufacturing automation. In the target use case, a factory operator gives a natural-language instruction to a local workcell assistant. A local SLM proposes a structured robot action, and a zero-trust validation layer evaluates whether the proposal is parseable, JSON-valid, schema-valid, semantically valid, safety-valid and execution-eligible. This separation is essential because a schema-valid output can still be semantically incorrect, unsafe, ambiguous or unsupported by the current workcell state.

Microsoft Foundry Local provides the local inference framing for the project. It offers a practical route for evaluating local model serving, privacy, offline resilience and deployment trade-offs without treating the model itself as a trusted robot controller [?]. The dissertation therefore evaluates Foundry Local as part of a local-first evidence chain, not as a production-certified robot-control platform.

The implemented system evolved through a series of prototypes, but Prototype 5 now acts as the final evidence orchestration and reporting layer. It consolidates prior evidence, records claim boundaries, generates the final claims matrix and evidence manifest, and adds final-stage evidence for model-family/precision behaviour, local-vs-cloud comparison and live local resource profiling. Prototype 1 is retained only as optional early feasibility context; its absence from core audit evidence is treated as a contextual documentation limitation rather than a missing proof for the final claims.

The current evidence baseline is complete for the dissertation pack. The Prototype 5 test suite passed, the final orchestrator completed, and the final dissertation pack generator completed. The evidence remains benchmark-based and prototype-scoped: it does not establish real-world robot safety, factory deployment readiness, clinical or production-grade assurance, or statistical generalisation beyond the evaluated command set.

1.2 Research Aim, Objectives and Questions

The main aim is to evaluate whether local SLMs served through Microsoft Foundry Local can support industrial robot task planning when model outputs are treated as untrusted action proposals and subjected to deterministic zero-trust validation before downstream execution.

The objectives are:

- (1) Evaluate the structured-output reliability of local SLMs on an industrial robot task-planning benchmark.
- (2) Distinguish JSON and schema validity from semantic validity, safety validity and execution eligibility.
- (3) Measure model-level false accepts and assess whether zero-trust validation prevents them becoming pipeline-level false accepts.
- (4) Compare local Foundry Local evidence with a cloud baseline under the same benchmark prompts.
- (5) Record local resource pressure and deployment trade-offs for the measured local inference path.
- (6) Produce an auditable dissertation evidence pack with explicit claim boundaries and reproducibility artefacts.

The main research question is:

Can local SLMs served through Microsoft Foundry Local support safe and reliable industrial robot task-planning workflows when their outputs are treated as untrusted action proposals and mediated through deterministic validation before execution?

The six sub-questions are:

- (1) How reliably can local SLMs generate parseable, JSON-valid and schema-valid industrial robot action plans?
- (2) To what extent does schema validity overestimate semantic correctness, safety validity and execution eligibility?
- (3) How does command ambiguity affect false accepts, rejection behaviour and execution eligibility?
- (4) What trade-offs appear across local model family and size in schema validity, latency, false accepts and execution eligibility?
- (5) How does local Foundry Local inference compare with cloud inference and live local resource profiling?
- (6) What is the safety-latency trade-off associated with stricter deterministic validation?

These questions are answered through the Prototype 3–5 evidence base. Prototype 1 is cited only as context, while Prototype 3, Prototype 4 and Prototype 5 provide the core benchmark, zero-trust, deployment and reporting evidence.

2 Background and Related Work

3 System Design and Methodology

3.1 System Architecture

The final architecture separates model generation from execution eligibility. This separation is the main design response to the finding that schema-valid output is not sufficient evidence of semantic correctness or safety. The local SLM is therefore not a robot controller; it is a proposal generator whose outputs must pass deterministic checks.

3.2 Layer 1: Local Inference

The first layer is the local inference layer. Natural-language commands are sent to a local SLM served through Microsoft Foundry Local. The model returns candidate structured action content. This

Table 1: Research questions mapped to detected evidence.

Research Question	Evidence Source	Metrics	Current Answer
Main RQ	Final claims matrix; Prototype 3–5 evidence	Schema validity, safety validity, execution eligibility	Local SLMs can support the workflow only as untrusted proposal generators mediated by deterministic validation.
RQ1	Final model comparison; Phi recovery summary	parse_success, json_valid, schema_valid	Local models produced structured proposals, but validity varied by model and evidence source.
RQ2	Model comparison; zero-trust comparison	schema_valid, execution_eligible, false accepts	Schema validity overestimated execution eligibility within the evaluated benchmark.
RQ3	Prototype 3 category evidence; Prototype 4 ambiguity summaries	ambiguity_level, rejection_reason, false accepts	Ambiguity is treated as a source of rejection and false-accept risk.
RQ4	Model comparison; run cards	Schema-valid rate, latency, false accepts	Model family and size affected reliability and latency within the detected evidence.
RQ5	Mode C and Mode D summaries	Local/cloud latency, JSON validity, CPU profile	Cloud was faster and more JSON-consistent in this run, while local inference preserved stronger privacy and offline-resilience properties.
RQ6	Safety-latency summary	False accepts, latency, rejection caveats	Stricter validation reduced unsafe false accepts in available records, with latency and rejection trade-offs requiring cautious interpretation.

layer is evaluated for request success, JSON validity, schema validity, latency and local deployment properties, but it is not trusted to decide whether a robot action should execute.

3.3 Layer 2: Zero-Trust Governance

The second layer is the zero-trust governance layer. It parses raw model output, checks JSON validity, applies schema validation, evaluates semantic validity where evidence exists, applies uncertainty or ambiguity handling, and then applies deterministic safety checks. A proposal becomes execution-eligible only if it passes the required gates under the defined validation policy. The dissertation uses the terms *model-level false accept* and *pipeline-level false accept* to distinguish a model proposal that appears acceptable before governance from an unsafe proposal that actually passes the full pipeline.

3.4 Layer 3: Optional Execution-Context Visualisation

The third layer is optional downstream execution-context visualisation. PyBullet, if added in future work, should be used only to visualise accepted and rejected proposals in a simulated workcell. It is not part of the core proof in this dissertation and must not be interpreted as real-world robot safety validation.

3.5 Evidence Logging

The evidence logger records model outputs, validation decisions, metrics, limitations and generated dissertation artefacts. Prototype 5 operationalises this reporting layer by producing the final evidence dashboard, claims matrix, limitations matrix, evidence manifest, safety-latency frontier and verification snapshots.

The Mermaid source for Figure 1 is stored at `figures/final_zero_trust_architecture.mmd`. If a rendered PDF is not available, the figure can be exported manually from Mermaid for Overleaf.

4 Methodology and Evaluation Framework

The methodology uses a controlled benchmark and a layered validation pipeline. The benchmark is designed for industrial robot

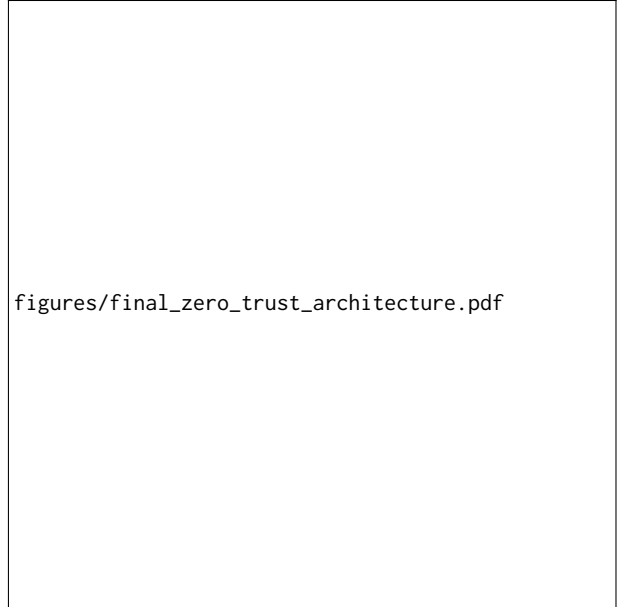


Figure 1: Local-first zero-trust evaluation architecture for industrial robot task-planning proposals.

task-planning proposals and includes ambiguity-stratified natural-language commands. The evaluation does not collapse performance into a single accuracy score. Instead, it records transport success, parse success, JSON validity, schema validity, semantic validity where evaluated, safety validity, execution eligibility, false accepts, false rejects, latency and resource pressure.

The validation pipeline follows a zero-trust sequence: raw model response, parser, JSON validity check, schema validator, semantic validator where available, uncertainty or ambiguity gate, deterministic safety gate, execution eligibility decision and evidence logging. This structure is intended to prevent model-level false accepts from becoming pipeline-level false accepts.

Model and run evidence are consolidated through the final evidence pack. Prototype 3 provides the main model-comparison and benchmark evidence. Prototype 4 provides zero-trust execution evidence, false-accept comparison and safety-latency evidence. Prototype 5 adds custom quantisation metadata, Phi-family response evidence, Mode C local-vs-cloud comparison, Mode D live resource profiling and the final evidence manifest.

The local-vs-cloud comparison is used to discuss deployment trade-offs rather than declare a universal winner. In the detected evidence, cloud inference was faster and more JSON-consistent, while local inference preserved stronger privacy and offline-resilience properties. Resource profiling records host-specific CPU and memory pressure for live local inference. These measurements support deployment discussion but are not hardware-independent cost models.

The safety-latency frontier summarises the trade-off between stricter validation and latency or rejection overhead. Where per-gate latency or rejection-rate data are not available, the evidence pack marks the values as missing or caveated rather than inferring them.

Reproducibility is supported through retained CSV, JSON, JSONL and Markdown evidence files, generated run cards, verification snapshots and a final evidence manifest. The final pack generator is deterministic and reads existing evidence files; it does not require cloud API calls or a running Foundry Local instance.

5 Prototype Evolution and Evidence Contribution

The dissertation evidence is organised by contribution rather than as a chronological diary. Earlier prototypes establish context and validation foundations, while the core claims rest on the Prototype 3–5 evidence base. This prevents the final report from depending on early feasibility artefacts that are not required for the final zero-trust evaluation claim.

6 Results and Evaluation

7 Results Overview

7.1 Foundry Local Inference Feasibility

The detected evidence indicates that local Foundry Local inference was feasible for the evaluated benchmark. Phi-family local evidence completed 30/30 requests. Mode D recorded 30/30 successful live Foundry Local requests. This supports feasibility for the measured local host and model, not for all deployment environments.

7.2 Structured-Output Reliability

The model comparison evidence shows that local models can produce structured robot action proposals, but reliability varies across model rows. JSON validity and schema validity are useful early gates, but neither is sufficient to establish semantic correctness, safety validity or execution eligibility.

7.3 Schema Validity Versus Execution Eligibility

Across the detected model-comparison rows, schema-valid rates exceeded execution-eligible rates. This supports the central dissertation claim that schema validity is not enough. A model response

may be structurally acceptable while still being semantically wrong, unsafe or unsuitable for execution under the validation policy.

7.4 Ambiguity and False Accepts

Ambiguity is treated as a risk factor in the evaluation. Prototype 3 difficulty/category evidence and Prototype 4 ambiguity summaries support the interpretation that ambiguous commands require clarification, rejection or stricter gating rather than direct execution. Per-ambiguity raw evidence may live in prior prototype repositories and should be cited with the caveat that availability can vary by checkout.

7.5 Model Family and Size Trade-Offs

The Qwen-family model rows show model-dependent differences in schema-valid rate, execution-eligible rate, false accepts and mean latency. These results indicate trade-offs within the evaluated benchmark, but they do not generalise to all SLM families, all model sizes or all hardware targets.

7.6 Zero-Trust Gating and False-Accept Prevention

The zero-trust comparison records a baseline trust mode with 76 unsafe false accepts and a zero-trust pipeline with 0 unsafe false accepts in the available execution records. This indicates that deterministic validation prevented model-level false accepts from becoming pipeline-level false accepts under the evaluated policy. It does not prove real-world robot safety.

7.7 Local-vs-Cloud Deployment Trade-Off

Mode C reports cloud mean latency of 1749.28 ms and local mean latency of 7028.0 ms. Cloud JSON-valid rate was 1.0, while local JSON-valid rate was 0.7333. In this benchmark, cloud inference was faster and more JSON-consistent, while local inference retained stronger privacy and offline-resilience properties. Cost was structurally discussed but not quantitatively measured.

7.8 Resource Profiling

Mode D recorded live Foundry Local resource profiling with mean latency 7896.97 ms and normalised mean CPU 48.31%. GPU and NPU counters were not detected, so no hardware acceleration claim is made. The resource profile is host-specific.

7.9 Safety-Latency Frontier

The safety-latency frontier indicates that stricter validation reduced unsafe false accepts in the available records. Some per-gate latency and rejection-rate values are not available in detected evidence, so the dissertation should discuss the frontier as bounded engineering evidence rather than a complete deployment optimisation curve.

7.10 Summary Against Research Questions

The results support the main research question within the evaluated benchmark: local SLMs can contribute to industrial robot task planning only when treated as untrusted proposal generators and mediated by deterministic validation. The evidence supports structured-output feasibility, highlights the insufficiency of schema

Table 2: Compressed metric taxonomy used in the evaluation framework.

Metric	Meaning	Why it matters
JSON validity	The response is syntactically valid JSON.	First machine-readable output gate.
Schema validity	The JSON conforms to the expected action schema.	Prevents malformed action proposals entering later stages.
Semantic validity	The action matches the intended command where explicitly evaluated.	Separates structural correctness from task correctness.
Safety validity	The proposal passes deterministic safety constraints.	Controls whether a proposal can proceed under the prototype policy.
Execution eligibility	The proposal passes all required gates.	Defines downstream eligibility without claiming real-world robot safety.
Latency and resource pressure	Measured timing and host resource behaviour.	Supports deployment feasibility discussion.

Table 3: Prototype contribution and evidence status.

Prototype	Purpose	Evidence Contribution	Status	Caveat
Prototype 1	Early local model-to-action feasibility context.	Optional audit/context evidence only.	Context only	Not core proof; does not support final safety or deployment claims.
Prototype 2	Deterministic validation and fail-closed foundation.	Schema and safety checks informing later validation stages.	Supporting	Does not by itself prove semantic safety under all robot tasks.
Prototype 3	Core Foundry Local benchmark and model evaluation.	Qwen-family model comparison, schema validity, execution eligibility, latency and model-level false accepts.	Core	Covers detected local models and benchmark scope only.
Prototype 4	Zero-trust execution and safety-latency evaluation.	Baseline versus zero-trust false accepts, execution-grounded safety evidence and extension summaries.	Core	Not physical robot safety certification.
Prototype 5	Final evidence orchestration and reporting layer.	Claims matrix, evidence manifest, local-vs-cloud comparison, live resource profiling, final dashboard and verification outputs.	Core reporting	Does not introduce new robot-control logic or Prototype 6.

validity, shows zero-trust false-accept prevention in available execution records, and documents local-vs-cloud and resource trade-offs. The claims remain prototype-scoped and benchmark-scoped.

8 Discussion

9 Alignment with the Microsoft IXN Brief

The project responds to the Microsoft IXN brief by evaluating local SLM deployment through Microsoft Foundry Local, positioning the work in an industrial use case, comparing local and cloud evidence, and recording latency, reliability, resource and deployment trade-offs. The alignment is evidence-based rather than promotional: requirements are marked as hit, partial or caveated according to detected artefacts.

10 Industrial Robotics Deployment Context

Industrial robotics and manufacturing automation provide a suitable domain for this work because robot task planning combines natural-language intent, structured action generation, safety constraints and deployment constraints. A factory operator may benefit from a local natural-language interface, but an industrial workcell cannot treat a language model as an authority. The cost of an unsafe or semantically wrong action is too high for direct model-to-execution control.

Local AI matters in this domain because manufacturers may require data locality, offline resilience, reduced cloud dependency and predictable operational boundaries. Microsoft Foundry Local fits this framing by allowing local model serving to be evaluated as part of an edge-oriented architecture. However, local execution

alone does not solve reliability or safety. It changes deployment properties, while deterministic validation remains necessary.

Schema validity is insufficient because it checks form rather than meaning. A proposal can contain the right fields and still select the wrong object, enter a restricted zone, ignore ambiguity or violate a safety rule. Zero-trust execution eligibility therefore becomes the central governance decision: a proposal is not execution-eligible until it passes the required parsing, schema, semantic, ambiguity and safety gates.

The deployment implication is that local SLMs are best framed as assistants that produce auditable proposals. The downstream system must log evidence, explain rejection reasons and fail closed when required. Before real factory deployment, the system would require a larger benchmark, scene-state integration, certified safety controls, operator workflow validation, hardware testing and assurance review. The present dissertation provides benchmark-based engineering evidence, not production safety certification.

11 Limitations and Claim Boundaries

The dissertation deliberately uses bounded language. Results indicate behaviour within the evaluated benchmark and validation policy; they do not establish production-grade safety, broad statistical generalisation or factory deployment readiness. Missing or unavailable values are not inferred.

12 Future Work

Future work should extend the evidence base without weakening the zero-trust separation between model generation and execution eligibility.

Table 4: Summary alignment with the Microsoft IXN brief.

Brief Requirement	Project Response	Evidence	Status
Industry use-case identification	Industrial robot task planning and manufacturing automation.	Industry use-case document; benchmark card.	HIT
Application/system architecture	Local SLM proposal generator plus deterministic zero-trust validation pipeline.	Architecture specification and Mermaid diagram.	HIT
Fully on-device Foundry Local prototype	Local Foundry Local inference path used for benchmark evidence.	Phi recovery summary; Mode D evidence summary.	HIT
Local LLM/SLM integration	Qwen-family and Phi-family local model evidence.	Model comparison and Phi evidence.	HIT
Latency benchmarking	Measured model, local, cloud and live profile latencies.	Model comparison; Mode C; Mode D.	HIT
Reliability benchmarking	Reliability decomposed into JSON, schema, semantic/safety and execution eligibility metrics.	Model comparison and zero-trust comparison.	HIT
Resource utilisation benchmarking	Live Foundry Local process profile.	Mode D resource profile.	HIT
Local-vs-cloud comparison	Same benchmark compared between local and cloud evidence.	Mode C local-vs-cloud summary.	HIT
Deployment feasibility discussion	Local-first trade-offs, resource pressure and safety-gate limits.	Claim boundaries and industry-use-case document.	HIT
Cost/maintenance trade-off	Local resource cost versus cloud dependency discussed qualitatively.	Mode C summary and claim boundaries.	PARTIAL
Reproducibility	Final evidence manifest, claims matrix, audit, final pack docs and tests.	Evidence manifest and final pack report.	HIT

Table 5: Claim boundaries for dissertation interpretation.

Claim Area	Supported Claim	Boundary/Caveat
Benchmark	Findings are reported within the evaluated benchmark.	The benchmark is small and controlled; it is not statistically powered for population-level inference.
Model	Detected local models can produce structured proposals.	The evidence does not cover every SLM family, size or hardware target.
Resource profiling	Mode D records host-specific live local resource pressure.	GPU/NPU counters were not detected, and results are hardware-specific.
Local-vs-cloud	Mode C supports deployment trade-off discussion.	It does not establish a universal local or cloud winner; cost was not quantitatively measured.
Simulation/PyBullet	PyBullet may be future visualisation context.	No PyBullet evidence is part of the core proof, and simulation would not prove physical safety.
Safety	Zero-trust reduced unsafe false accepts in available records.	This is not real-world robot safety certification.
Generalisability	Claims are bounded to the evaluated command set and policy.	No statistical generalisation beyond the benchmark is claimed.
Evidence availability	Prototype 3–5 provide core evidence; Prototype 1 is context-only.	Missing Prototype 1 audit files are contextual, not missing core proof.

- **PyBullet visual execution-context demonstrator:** add an optional visual layer that shows execution-eligible proposals moving in a simulated workcell and rejected proposals producing no motion with a logged reason. This should remain visualisation, not core proof.
- **Safety policy DSL:** define a more expressive policy language for zones, objects, tool states, operator permissions and task constraints.
- **Clarification recovery loop:** extend ambiguity handling into a live operator clarification workflow while preserving fail-closed behaviour.
- **Hybrid local/cloud router:** evaluate when local inference, cloud inference or abstention is appropriate under latency, privacy, cost and reliability constraints.
- **Larger benchmark:** expand command count, ambiguity strata, object types, scene states and manipulation tasks.
- **Hardware matrix:** run the evaluation across CPU, GPU, NPU and Copilot+ PC-class hardware to compare resource pressure and latency.
- **Higher-fidelity simulator or real robot validation:** test the validation policy against richer robot state and physical constraints before making deployment claims.

- **Formal safety and assurance case:** develop a structured assurance case linking hazards, controls, evidence and residual risk.

13 Conclusion

14 Conclusion

This report has presented the current evidence baseline for a local-first zero-trust evaluation framework for industrial robot task planning using Microsoft Foundry Local. The core finding is that schema validity is not sufficient for execution eligibility: local SLMs can generate structurally valid action proposals that still require deterministic validation before they can safely reach any downstream execution context. Prototype 5 now provides the final evidence orchestration layer, consolidating benchmark evidence, zero-trust validation results, local-vs-cloud comparison, resource profiling, claim boundaries and verification artefacts. The work remains benchmark-scoped and prototype-scoped, but it provides a coherent foundation for the final dissertation and for future extensions such as policy-driven validation, clarification recovery and simulated execution-context visualisation.

Acknowledgements

This work was conducted as part of the UCL Microsoft IXN dissertation project.

References

A Verification Evidence

This appendix records the verification evidence for the final writing pack. These outputs confirm that the reporting layer can be regenerated from existing evidence files; they are not new model-evaluation results.

- Test command:python -m pytest tests/prototype5 -v.
Key output:===== 62 passed
in 0.97s =====.
- Orchestrator command:python -m src.prototype5.run_orchestrator_keyoutput:
Prototype 5 Final Evidence Orchestrator: COMPLETE.

- Final pack generator command: python -m src.prototype5.generator
Prototype 5 final dissertation pack generated..

The raw verification snapshots are stored at:

- results/prototype5/verification_snapshots/pytest_prototype5.
- results/prototype5/verification_snapshots/orchestrator_output
- results/prototype5/verification_snapshots/final_pack_generator
- results/prototype5/verification_snapshots/verification_summary

The generated evidence files include the final evidence dashboard, claims matrix, dissertation metrics, evidence manifest, Microsoft brief alignment matrix, safety-latency frontier and final pack completion report. Remaining caveats include the absence of production robot safety validation, no statistical generalisation beyond the benchmark, no quantitative cloud cost accounting, no detected GPU/NPU counters in Mode D, and Prototype 1 being context-only evidence.

References